
SC1 Analytics

Full Technical Documentation

Retail1 Project — CCTV Person Analytics Pipeline
Jetson Orin NX 16 GB • DeepStream 7.1 • TensorRT 10.3

Platform	NVIDIA Jetson Orin NX 16 GB, L4T R36.4.7
CUDA	12.6
TensorRT	10.3.0.30
DeepStream	7.1.0
GStreamer	1.20.3
PyTorch	2.8.0 (NVIDIA Jetson wheel)
OpenCV	4.13.0
pyds	1.2.0 (cp310-aarch64)

WiseVision — July 2026

Contents

1. System Overview

1.1 What Was Built

SC1 is a real-time retail analytics pipeline running on a single NVIDIA Jetson Orin NX 16 GB edge device. It ingests 1080p/25 fps camera streams, performs per-person detection, cross-frame tracking, role classification (seller vs. client), store-entry counting, and seller-client interaction (PEC) timing — all without cloud dependencies, without face recognition, and without any model retraining at deployment time.

The system is composed of two codebases:

- `Final version_-Ahmed/SC1.py` — the team’s standalone Python prototype (YOLO11m via Ultralytics + BoT-SORT via boxmot). Used for algorithm design, validation, and ground-truth comparison.
- `deepstream-sc1-native/` — The production DeepStream port: identical algorithm logic running inside a GStreamer/TensorRT/NvDCF pipeline. This is what goes to stores.

1.2 Validated Metrics

The pipeline was run on `CRK01.mp4`: 18 903 frames, 25 fps, 756 s (12.6 min), 1080p, MPEG-TS container + H.265 codec (despite the `.mp4` extension).

Table 1.1: Validated performance on CRK01.mp4 (Store 1 rush-hour footage)

Metric	SC1.py (prototype)	DeepStream port
End-to-end throughput	7.3 FPS	43.3 FPS
Wall-clock (12.6 min video)	21 m 12 s	7 m 17 s
vs. real-time	0.59×	1.73×
Speedup vs. prototype	—	5.9×
Probe logic only	—	10.0 ms/frame
PEC events detected	4 / 4	4 / 4
Average PEC duration	50.9 s	51.1 s
Clients entered (zone)	—	2 / 4 (zone calibration gap)
GPU utilisation	—	98–99%

1.3 Repository Structure

```

1  deepstream-sc1-native/
2  src/
3      pipeline.py          # GStreamer pipeline, probes, main entry
                             point
4      sc1_logic.py         # Business logic (framework-agnostic, pure
                             numpy/cv2)
5  configs/
6      config_infer_primary_yolo11m_sc1.txt  # nvinfer (PGIE) config
7      config_tracker_sc1.yml               # NvDCF + ReID tracker
                             config
8  models/
9      yolo11m/
10         yolo11m.onnx          # 77 MB   (YOLO11m with
                             DeepStreamOutput)
11         yolo11m_b1_gpu0_fp16.engine      # 46 MB   (TRT FP16,
                             built on first run)
12         labels.txt
13     reid/
14         export_osnet_onnx.py          # One-off OSNet export
                             script
15         osnet_ain_x1_0_msmt17.onnx      # 8.4 MB
16         osnet_ain_x1_0_msmt17.onnx_b16_gpu0_fp16.engine  # 5.7 MB
17  DeepStream-Yolo/
18     nvdsinfer_custom_impl_Yolo/
19         libnvdsinfer_custom_impl_Yolo.so  # Custom C++ bbox parser
                             (compiled)
20         nvdsparsebbox_Yolo.cpp          # NvDsInferParseYolo
                             implementation
21         nvdsinfer_yolo_engine.cpp        #
22         NvDsInferYoloCudaEngineGet hook
23         yolo.cpp / yolo.h              # Model descriptor
24         yoloForward*.cu                # CUDA forward-pass
25     kernels
26  Final version_-Ahmed/

```

```
26 SC1.py                                # Ahmed's original
   prototype
27 output/
28   CRK01_SC1_output.mp4                # Prototype annotated
   output (1.1 GB)
```

Listing 1.1: Project layout

2. Hardware Platform

2.1 Jetson Orin NX 16 GB

Table 2.1: Hardware specification

Field	Value
Module	NVIDIA Jetson Orin NX 16 GB
SoC	NVIDIA Orin (ARMv8.2-A Cortex-A78AE $\times 8$)
GPU	NVIDIA Ampere, 1024 CUDA cores, 32 Tensor Cores
DLA	2 $\times$ Deep Learning Accelerator @ 614 MHz
NVDEC	1 $\times$ NVDEC (H.264 / H.265 / VP9 / AV1)
NVENC	1 $\times$ NVENC
PVA	1 $\times$ Programmable Vision Accelerator
DRAM	16 GB LPDDR5 unified (CPU + GPU share same pool)
L4T	R36.4.7
JetPack	6.x

2.2 GPU Architecture (Ampere)

The Orin NX GPU is an NVIDIA Ampere-class device. Key structural properties relevant to this pipeline:

- **CUDA cores:** 1024, organised in 8 Streaming Multiprocessors (SMs). Used by TensorRT for inference and by the NvDCF tracker’s correlation filters.
- **Tensor Cores:** 32 (second-generation, Ampere). Used implicitly by TensorRT during FP16 matrix multiplications (GEMM layers in YOLO11m and OSNet).
- **Shared L2 cache:** 2 MB. Shared across all SMs; TRT batched inference exploits spatial locality in feature maps.
- **INT8 / FP16 support:** Full. The pipeline uses FP16 throughout (`network-mode=2` in `nvinfer`, `networkMode=1` in the tracker).

2.3 Dedicated Compute Engines

2.3.1 NVDEC — Hardware Video Decoder

NVDEC is a dedicated fixed-function silicon block separate from the CUDA cores. It decodes compressed bitstreams (H.264, H.265, VP9, AV1) directly into NVMM (GPU-accessible) memory without involving the CPU or the CUDA cores at any point.

In this pipeline, `decodebin` selects `nvv4l2decoder` automatically on Jetson for any supported codec. The source video is H.265 @ 25 fps. NVDEC consumes the compressed bitstream and produces raw NV12 frames in NVMM.

2.3.2 DLA — Deep Learning Accelerator

Both DLA cores (DLA0, DLA1) are **online at 614MHz** but **not used** by the current pipeline. `nvinfer` runs exclusively on the GPU. DLA offload (`enable-dla=1`, `use-dla-core=0` in the `nvinfer` config) would free GPU compute headroom for additional camera streams.

2.3.3 PVA — Programmable Vision Accelerator

Unused in the current pipeline. PVA is suited for classical CV preprocessing (resize, normalise, warp) and would be the target for future optimisation of the probe-side `cv2.cvtColor` operations.

2.4 Unified Memory Architecture

Unlike discrete GPU systems, Jetson has a single physical DRAM pool shared by CPU and GPU. NVMM (NVIDIA Memory Management) buffers are allocated once in this shared pool and accessed by NVDEC, the CUDA cores, and the CPU probe without any PCI-e transfer. However, the GPU and CPU maintain separate MMU mappings and cache coherency domains.

When a probe calls `pyds.get_nvds_buf_surface()`, it inserts a CPU mapping into the NVMM allocator's page table and flushes GPU caches. This is not a memory copy — it is a TLB/cache-coherency operation. The cost is non-zero but far below a discrete GPU `memcpy`.

2.5 Clock Configuration

For all validated runs:

```

1 sudo nvpmode1 -m 0           # MAXN power mode (no throttling)
2 sudo jetson_clocks          # Lock all clocks to maximum frequency

```

Listing 2.1: Clock configuration commands

Table 2.2: Key clocks at MAXN + jetson_clocks

Subsystem	Max Freq	Notes
GPU	918 MHz	CUDA cores + Tensor Cores
CPU (all)	2201.6 MHz	8× Cortex-A78AE
DLA	614.4 MHz	Both cores, unused
EMC	3199 MHz	DRAM bus; 204 MHz observed (demand-governed)

EMC behaviour: The External Memory Controller frequency is governed by actual memory bandwidth demand, not GPU occupancy. At 204 MHz during inference runs, the bottleneck is the CUDA compute pipeline, not DRAM bandwidth. EMC scales up automatically under sustained memcpy or frame-buffer-intensive loads.

3. Analytics Algorithm — SC1

3.1 Overview

The SC1 algorithm processes a stream of $(f, \mathbf{D}_f)$ pairs, where f is a decoded BGR frame and $\mathbf{D}_f = \{(b_i, c_i, t_i)\}$ is the set of tracked person detections at frame f with bounding box $b_i \in \mathbb{R}^4$, confidence $c_i \in [0, 1]$, and tracker ID $t_i \in \mathbb{Z}_{>0}$.

Per frame the algorithm produces:

1. A classified set of persons $\mathcal{P}_f = \{(b_i, r_i)\}$ with role $r_i \in \{\text{SELLER}, \text{CLIENT}\}$.
2. Incremented entry counter N_{entry} .
3. Incremented PEC counter N_{PEC} and updated state machine.

All logic lives in `src/sc1_logic.py` as the `SC1State` class, called from the DeepStream tracker pad probe. The same code ran unchanged inside `SC1.py`'s `main()` loop.

3.2 Detection: YOLO11m

- **Model:** YOLO11m (Ultralytics) trained on COCO (80 classes).
- **Input:** 640×640 px, RGBA→normalised float (net-scale-factor = $1/255 \approx 0.003922$).
- **Class filter:** All 79 non-person classes filtered out via `filter-out-class-ids` in the `nvinfer` config. Only class 0 (person) passes to the tracker.
- **Detection threshold:** `DETECT_CONF` = 0.1 — low threshold to avoid missing partially visible staff.
- **Client confidence floor:** `CONF_CLIENT` = 0.2 applied *downstream* in the probe (not in `nvinfer`) to discard low-confidence client boxes while keeping all staff detections regardless of confidence.
- **NMS:** IoU threshold = 0.5, top- k = 300 proposals.
- **Aspect ratio:** Maintained with symmetric padding to avoid spatial distortion.

3.3 Multi-Object Tracking

3.3.1 Prototype (SC1.py): BoT-SORT + OSNet ReID

In the Python prototype, tracking is handled by boxmot’s BoT-SORT implementation:

- Two-stage matching: high-confidence detections associated first via IoU; low-confidence detections matched to lost tracks via appearance.
- Camera motion compensation via ECC (Enhanced Correlation Coefficient) on every processed frame (every 3rd decoded frame, SKIP=3).
- Per-track appearance EMA: $\alpha = 0.95$ (patched via `_patch_appearance_ema()`), meaning historical appearance is heavily weighted to resist single-frame misidentification.
- OSNet feature extraction on every processed frame (`reidExtractionInterval=1`).

3.3.2 DeepStream Port: NvDCF + ReID

DeepStream’s `nvtracker` element loads `libnvds_nvmultiobjecttracker.so`, which implements the NvDCF (NVIDIA Deep Correlation Filter) tracker. BoT-SORT is not available as a native DeepStream plugin — it would require a C++ reimplementation of the `NvDsTracker` C API (`nvdstracker.h`).

NvDCF approximates BoT-SORT’s functionality as follows:

Table 3.1: BoT-SORT to NvDCF parameter mapping

BoT-SORT parameter	Value	NvDCF equivalent
TRACK_LOW_THRESH	0.1	<code>minDetectorConfidence</code>
NEW_TRACK_THRESH	0.2	<code>tentativeDetectorConfidence</code>
TRACK_HIGH_THRESH	0.4	<code>minTrackerConfidence</code>
MATCH_THRESH	0.8	<code>minMatchingScore4overall</code>
APPEARANCE_THRESH	0.3	<code>minMatchingScore4VisualSimilarity</code>
TRACK_BUFFER	180 proc. frames (≈ 21.6 s)	<code>maxShadowTrackingAge=540</code>

TRACK_BUFFER conversion: SC1.py processes every 3rd frame (SKIP=3) at 25 fps, giving an effective rate of ≈ 8.33 fps. 180 processed frames = 21.6 s real time. The DeepStream pipeline processes every decoded frame (25 fps), so the equivalent shadow age is $21.6 \times 25 = 540$ frames.

3.4 Seller / Client Classification

3.4.1 HSV Colour Band (UniformModel)

The classifier identifies staff by the specific colour of their uniform (beige/tan lower body + white upper body). The colour band is *baked in* at deployment time — no reference images are needed at runtime.

Baked-in band:

$$\text{UNIFORM_BAND_LO} = [18, 0, 51]_{HSV}, \quad \text{UNIFORM_BAND_HI} = [49, 54, 240]_{HSV}$$

Lower-body crop region (pants):

$$y_{lo} = y_1 + 0.55(y_2 - y_1), \quad y_{hi} = y_1 + 0.88(y_2 - y_1), \quad x_{\text{margin}} = 0.25(x_2 - x_1)$$

Upper-body crop region (shirt):

$$y_{lo} = y_1 + 0.15(y_2 - y_1), \quad y_{hi} = y_1 + 0.45(y_2 - y_1)$$

A detection is classified as SELLER if both conditions hold simultaneously:

$$\text{color_ratio} = \frac{|\{p \in \text{pants_crop} : p \in \text{band}\}|}{|\text{pants_crop}|} \geq 0.30 \quad (3.1)$$

$$\text{white_ratio} = \frac{|\{p \in \text{shirt_crop} : S_p \leq 80 \wedge V_p \geq 85\}|}{|\text{shirt_crop}|} \geq 0.25 \quad (3.2)$$

3.4.2 SEED Window Bootstrap

```

1 SEED_SECONDS = 3.0
2
3 seeding = timestamp <= SEED_SECONDS
4 if seeding:
5     role = "SELLER"
6     self.uniform.add_seed_sample(frame, bbox)
7     if tid is not None:
8         self.seller_tracks.add(tid)

```

Listing 3.1: Seed window in SC1State.process_frame()

Every person visible in the first 3 seconds is assumed to be staff (clients do not enter before store opening). This bootstraps the `seller_tracks` set without any manual labelling.

3.4.3 Anti-Flicker Voting

A track is only promoted to SELLER after matching the colour band in `SELLER_VOTES = 2` independent frames. Once promoted, the track ID is permanently added to `seller_tracks` — it never reverts.

```

1 if is_unif:
2     self.seller_votes[tid] = self.seller_votes.get(tid, 0) + 1
3     if self.seller_votes[tid] >= SELLER_VOTES: # >= 2
4         self.seller_tracks.add(tid)
5 role = "SELLER" if tid in self.seller_tracks else "CLIENT"

```

Listing 3.2: Vote-based promotion

3.4.4 Staff Desk Zone Override

A person whose bounding box overlaps $\geq 50\%$ with a SELLER_ZONE polygon is hard-classified as SELLER regardless of colour. This handles seated staff whose lower body (uniform) is hidden behind the counter.

Sampling method: $6 \times 6 = 36$ grid points are tested across the bounding box using `cv2.pointPolygonTest`.

```

1 SELLER_ZONE_FRAC = 0.50 # 50% of box must be inside zone
2
3 for px in np.linspace(x1, x2, 6):
4     for py in np.linspace(y1, y2, 6):
5         total += 1
6         if any(cv2.pointPolygonTest(poly, (px, py), False) >= 0
7             for poly in _SELLER_POLYS):
8             inside += 1
9 return (inside / total) >= SELLER_ZONE_FRAC

```

Listing 3.3: Grid-sampled zone overlap test

3.5 Zone System

All zones are defined as pixel-space polygons specific to CRK01’s camera view.

Table 3.2: Zone types and their function

Zone type	Count	Function
MIRROR_ZONES	3	Drop detections that are camera reflections
IGNORE_ZONES	1	Drop static false positives (e.g. handbag on shelf)
SELLER_ZONES	3	Force SELLER role for persons substantially inside
ENTREE_ZONE	1	Gate for directional entry counting

MIRROR and IGNORE zones are applied in Probe 1 (before the tracker) via `pyds.nvds_remove_obj_meta`. SELLER and ENTREE zones are applied in Probe 2 (after the tracker), where track IDs are available.

3.6 Entry Counting

A client track is counted as “entered” when all four conditions hold simultaneously:

1. Its feet (bounding-box bottom-centre (f_x, f_y)) are inside `ENTREE_ZONE`.
2. Its track ID has not been counted before ($t_i \notin \text{entered_ids}$).
3. The role is `CLIENT`.
4. It is moving *inward* (towards the store interior).

Directional check uses a signed perpendicular distance to a reference line L (defined by `LINE1_START`, `LINE1_END`):

$$d(P, L) = \frac{(L_e - L_s) \times (P - L_s)}{\|L_e - L_s\|} = \frac{\Delta x \cdot (P_y - L_{s,y}) - \Delta y \cdot (P_x - L_{s,x})}{\sqrt{\Delta x^2 + \Delta y^2}}$$

The track is counted only if $\text{ENTER_SIGN} \cdot (d_{\text{current}} - d_{\text{previous}}) \geq 1 \text{ px}$.

3.7 PEC State Machine

3.7.1 State Definitions

The PEC (Point d’Engagement Client) state machine tracks one engagement at a time using three implicit states encoded in two boolean flags:

Table 3.3: PEC state encoding

State	pec_active	pec_counted
IDLE	False	False
ACTIVE	True	False
COUNTED	True	True

3.7.2 Proximity Detection

A contact is detected when the bounding-box gap between a `CLIENT` and any `SELLER` is below `PEC_PROX_DIST` = 350 px:

$$\text{gap} = \sqrt{\max(0, x_{c,1} - x_{s,2}, x_{s,1} - x_{c,2})^2 + \max(0, y_{c,1} - y_{s,2}, y_{s,1} - y_{c,2})^2} < 350$$

3.7.3 State Transitions

```

1  PEC_MIN_SECS    = 8.0    # minimum engagement duration before
    counting
2  PEC_GRACE_SECS  = 12     # gap tolerance before closing engagement
3  PEC_FOLLOW_DIST = 160    # max position drift for "same client"
    tracking
4  PEC_EMA         = 0.5    # position smoothing factor
5
6  if self.pec_active:
7      # find the closest contact to the anchored position
8      best = None; best_d = PEC_FOLLOW_DIST
9      for ccx, ccy in contacts:
10         dd = np.hypot(ccx - self.pec_pos[0], ccy - self.pec_pos[1])
11         if dd < best_d:
12             best_d, best = dd, (ccx, ccy)
13
14         if best is not None:                # served client
15             # EMA-smooth the position anchor
16             self.pec_pos = (PEC_EMA * self.pec_pos[0] + (1-PEC_EMA) *
17                             best[0],
18                             PEC_EMA * self.pec_pos[1] + (1-PEC_EMA) *
19                             best[1])
20             self.pec_last_t = timestamp
21
22             # ACTIVE -> COUNTED transition
23             if not self.pec_counted and \
24                 timestamp - self.pec_start_t >= PEC_MIN_SECS:
25                 self.pec_count += 1
26                 self.pec_counted = True
27
28             elif timestamp - self.pec_last_t > PEC_GRACE_SECS: # grace
29                 # period elapsed
30                 if self.pec_counted:
31                     self.pec_durations.append(self.pec_last_t - self.
32                     pec_start_t)
33                     self.pec_active = False; self.pec_pos = None # -> IDLE
34
35 else:
36     # IDLE
37     if contacts:                # -> ACTIVE
38         self.pec_active = True; self.pec_counted = False
39         self.pec_pos = served    # anchor position
40         self.pec_start_t = timestamp
41         self.pec_last_t = timestamp

```

Listing 3.4: PEC state machine (from SC1State.process_frame())

3.7.4 Position EMA Anchoring

The served client is tracked by *position*, not track ID. This makes the engagement robust to BoT-SORT / NvDCF track-ID churn at occlusion boundaries. The EMA factor $\alpha = 0.5$ provides moderate smoothing — the anchor moves with the client but resists sudden jumps from brief occlusions.

3.7.5 Grace Period

The 12-second grace period (**PEC_GRACE_SECS**) allows the served client to be temporarily occluded (another customer walking between the seller and the camera) without terminating the engagement. Only if the client is absent for more than 12 continuous seconds is the engagement closed.

4. DeepStream Pipeline Architecture

4.1 GStreamer Primer (relevant to this pipeline)

DeepStream is built on GStreamer 1.x. A pipeline is a directed acyclic graph of *elements* connected by *pads*. Data flows as *buffers* attached to pad events. *Probe callbacks* (pad probes) are functions registered on a pad that intercept every buffer — they can read and modify both the raw buffer bytes and the attached DeepStream metadata (`NvDsBatchMeta`).

All NVIDIA elements exchange buffers in `memory:NVMM` format — a GStreamer memory allocator backed by NVMM-managed GPU memory. Non-NVMM conversions (e.g. `appsink` to system RAM) require explicit conversion elements.

4.2 Full Element Chain

```
1  filesrc
2  |
3  +-- [path: location=CRK01.mp4]
4  |
5  decodebin    [auto-detects MPEG-TS container + H.265 codec]
6  |            [selects: tsdemux -> h265parse -> nvv4l2decoder]
7  |            [output: NV12, memory:NVMM]
8  |            (pad-added callback links decoded video pad to streammux)
9  |
10 nvstreammux  [batches frames; batch-size=1, width=1920, height=1080]
11 |            [batched-push-timeout=40000 us, live-source=0]
12 |            [output: NV12, memory:NVMM, batch metadata attached]
13 |
14 nvvideoconvert [NV12 -> RGBA, stays in memory:NVMM]
15 |
16 capsfilter    [video/x-raw(memory:NVMM), format=RGBA]
17 |
18 nvinfer       [PGIE: YOLO11m TRT FP16 engine]
19 |            [attaches NvDsObjectMeta per detection to NvDsBatchMeta]
20 |
21 +--- [PROBE 1: pgie_src_pad_buffer_probe]
```

```

22 | [reads NvDsObjectMeta, removes MIRROR/IGNORE zone detections
23 | [calls: pyds.nvds_remove_obj_meta_from_frame()]
24 |
25 nvtracker [NvDCF + OSNet ReID, ll-lib=libnvds_nvmultiobjecttracker
    .so]
26 | [adds object_id (track ID) to each NvDsObjectMeta]
27 |
28 +--- [PROBE 2: tracker_src_pad_buffer_probe]
29 | [maps NVMM buffer to CPU, runs SC1State.process_frame()]
30 | [writes annotated frame back to NVMM buffer]
31 |
32 nvvideoconvert [RGBA -> NV12, memory:NVMM]
33 |
34 capsfilter [video/x-raw(memory:NVMM), format=NV12]
35 |
36 nvv4l2h264enc [H.264 encode via NVENC; bitrate=8 Mbps]
37 |
38 h264parse
39 |
40 qtmux
41 |
42 filesink [location=output/CRK01_deepstream_output.mp4, sync=False]

```

Listing 4.1: GStreamer pipeline topology

4.3 Source & Decode

```

1 filesrc = Gst.ElementFactory.make("filesrc", "filesrc")
2 filesrc.set_property("location", args.input)
3
4 # decodebin auto-detects the demux/parse/decode chain.
5 # CRK01.mp4 is actually MPEG-TS + H.265 despite the .mp4 extension.
6 # Both qtdemux and tsdemux fail on it directly.
7 # decodebin selects: tsdemux -> h265parse -> nvv4l2decoder (NVDEC
   hardware)
8 decodebin = Gst.ElementFactory.make("decodebin", "decodebin")
9
10 def on_decodebin_pad_added(dbin, pad):
11     caps = pad.get_current_caps() or pad.query_caps()
12     s = caps.get_structure(0)
13     if s.get_name().startswith("video/"):
14         pad.link(mux_sink) # connect decoded video to
   streammux
15
16 decodebin.connect("pad-added", on_decodebin_pad_added)

```

Listing 4.2: Source and decode element setup

Why decodebin instead of explicit demux + decoder: The source files use MPEG-TS container with H.265 video but carry a .mp4 extension. `file` reports them as raw “data”; `qtdemux` and `tsdemux` both reject them without `decodebin`’s auto-negotiation.

`decodebin` uses `typefind` and caps negotiation to select the correct path regardless of file extension.

4.4 Muxing (`nvstreammux`)

```
1 streammux.set_property("width", 1920)
2 streammux.set_property("height", 1080)
3 streammux.set_property("batch-size", 1)
4 streammux.set_property("batched-push-timeout", 40000) #
  microseconds
5 streammux.set_property("live-source", 0) # 0=file; set to 1 for
  RTSP
6
7 mux_sink = streammux.get_request_pad("sink_0")
```

Listing 4.3: Streammux configuration

`nvstreammux` aggregates one or more decoded streams into a batch. Batch size is 1 for the current single-camera deployment. The element creates `NvDsBatchMeta` metadata, which all downstream elements and probes read.

4.5 Format Conversion: RGBA for Probe Access

```
1 conv_in = Gst.ElementFactory.make("nvvideoconvert", "conv_in")
2 caps_rgba = Gst.ElementFactory.make("capsfilter", "caps_rgba")
3 caps_rgba.set_property("caps",
4   Gst.Caps.from_string("video/x-raw(memory:NVMM), format=RGBA"))
```

Listing 4.4: Pre-inference conversion to RGBA

RGBA is required because `pyds.get_nvds_buf_surface()` returns a numpy view of the buffer in RGBA plane layout. The subsequent `cv2.cvtColor` call converts `RGBA→BGR` for HSV analysis. NV12 is used for encode-output only (colour space standard for H.264/H.265 encoders).

4.6 Primary Inference (`nvinfer` / `YOLO11m`)

```
1 pgie = Gst.ElementFactory.make("nvinfer", "pgie")
2 pgie.set_property("config-file-path", PGIE_CONFIG)
3 # All inference parameters (model path, FP16, thresholds) in config
  file
```

Listing 4.5: `nvinfer` element setup

`nvinfer` loads the TensorRT FP16 engine, runs inference on the batch, and attaches one `NvDsObjectMeta` per detection to the frame's `NvDsFrameMeta`. Key metadata fields populated:

Table 4.1: NvDsObjectMeta fields used by downstream probes

Field	Content
<code>rect_params.left</code>	Bounding box left (pixels)
<code>rect_params.top</code>	Bounding box top (pixels)
<code>rect_params.width</code>	Bounding box width (pixels)
<code>rect_params.height</code>	Bounding box height (pixels)
<code>confidence</code>	Detection confidence $\in [0, 1]$
<code>class_id</code>	Always 0 (person only passes nvinfer filter)
<code>object_id</code>	Populated by nvtracker (not nvinfer)

4.7 Probe 1: Mirror / Ignore Zone Filter

4.7.1 Registration and Buffer Access

```

1  pgie_src_pad = pgie.get_static_pad("src")
2  pgie_src_pad.add_probe(
3      Gst.PadProbeType.BUFFER,
4      pgie_src_pad_buffer_probe,
5      0)

```

Listing 4.6: Probe 1 registration on PGIE src pad

The probe fires on every buffer leaving `nvinfer` *before* the buffer reaches `nvtracker`. This mirrors SC1.py’s Step 1b, where mirror and ignore zones are applied before BoT-SORT processes the detections — the tracker never sees phantom reflections or static false positives.

4.7.2 Metadata Traversal and Object Removal

```

1  def pgie_src_pad_buffer_probe(pad, info, u_data):
2      gst_buffer = info.get_buffer()
3      if not gst_buffer:
4          return Gst.PadProbeReturn.OK
5
6      # Access the batch metadata attached to the GStreamer buffer
7      batch_meta = pyds.gst_buffer_get_nvds_batch_meta(hash(
8          gst_buffer))
9
10     l_frame = batch_meta.frame_meta_list
11     while l_frame is not None:
12         frame_meta = pyds.NvDsFrameMeta.cast(l_frame.data)
13
14         to_remove = []
15         l_obj = frame_meta.obj_meta_list
16         while l_obj is not None:
17             obj_meta = pyds.NvDsObjectMeta.cast(l_obj.data)
18             rp = obj_meta.rect_params

```

```

18         bbox = (rp.left, rp.top, rp.left + rp.width, rp.top +
19                 rp.height)
20         if sc1_logic.in_mirror_zone(bbox) or sc1_logic.
21         in_ignore_zone(bbox):
22             to_remove.append(obj_meta)
23             l_obj = l_obj.next
24         # Remove after full traversal (modifying list during
25         # iteration is unsafe)
26         for obj_meta in to_remove:
27             pyds.nvds_remove_obj_meta_from_frame(frame_meta,
28             obj_meta)
29         l_frame = l_frame.next
30         return Gst.PadProbeReturn.OK

```

Listing 4.7: Probe 1 implementation

Traversal-then-delete pattern: The object list must be fully traversed before removing any element. `pyds.nvds_remove_obj_meta_from_frame` modifies the linked list; calling it mid-traversal corrupts `l_obj.next`.

4.8 Tracker (nvtracker / NvDCF + ReID)

```

1 tracker = Gst.ElementFactory.make("nvtracker", "tracker")
2 tracker.set_property("tracker-width", 640)
3 tracker.set_property("tracker-height", 384)
4 tracker.set_property("ll-lib-file",
5                       "/opt/nvidia/deepstream/deepstream-7.1/lib/
6                       libnvds_nvmultiobjecttracker.so")
7 tracker.set_property("ll-config-file", TRACKER_CONFIG)
8 tracker.set_property("gpu-id", 0)

```

Listing 4.8: nvtracker element configuration

The tracker operates internally at 640×384 (the resolution at which NvDCF correlation filters and ReID crops are extracted). This is independent of the full-resolution NVMM buffer which flows through the pipeline unchanged.

4.8.1 ReID Sub-Pipeline inside NvMultiObjectTracker

For each tracked object at `reidExtractionInterval=1` frames, the tracker:

1. Crops the bounding box region from the NVMM buffer.
2. Resizes to 256×128 (OSNet input: $3 \times 256 \times 128$, NCHW).
3. Subtracts ImageNet mean per channel: $[123.675, 116.28, 103.53]$, applies scale $\approx 1/(255 \cdot \bar{\sigma}) = 0.01735$.

4. Runs the OSNet TRT FP16 engine in batch mode (batch-size=16).
5. Produces 512-dimensional L2-normalised feature vectors.
6. Uses cosine similarity between gallery and query features for re-association of lost tracks (`reidType=2`).

4.9 Probe 2: SC1 Business Logic

This is the core of the production pipeline. The probe fires on every buffer leaving `nvtracker` and is responsible for all SC1 analytics.

4.9.1 NVMM Buffer Access

```

1 # Map NVMM buffer to a CPU-accessible numpy array
2 # This is NOT a copy -- it inserts a CPU TLB mapping into the NVMM
  allocator
3 n_frame = pyds.get_nvds_buf_surface(hash(gst_buffer), frame_meta.
  batch_id)
4 # n_frame is a numpy array: shape (1080, 1920, 4), dtype uint8,
  format RGBA
5 # Memory layout: RGBA plane, row-major, C-contiguous
6
7 # Convert RGBA -> BGR for OpenCV HSV operations (CPU operation)
8 frame_bgr = cv2.cvtColor(n_frame, cv2.COLOR_RGBA2BGR)

```

Listing 4.9: NVMM buffer CPU mapping

The `hash(gst_buffer)` call extracts the buffer’s memory address as a Python integer, which `pyds` uses to locate the `NvBufSurface` struct. The returned numpy array is a *view* into the mapped NVMM memory — writes to it are reflected in the GPU buffer.

4.9.2 Detection Metadata Harvest

```

1 dets = []
2 l_obj = frame_meta.obj_meta_list
3 while l_obj is not None:
4     obj_meta = pyds.NvDsObjectMeta.cast(l_obj.data)
5     rp = obj_meta.rect_params
6     bbox = np.array([rp.left, rp.top,
7                     rp.left + rp.width, rp.top + rp.height])
8     conf = float(obj_meta.confidence)
9     tid = int(obj_meta.object_id) # assigned by nvtracker
10    dets.append((bbox, conf, tid))
11    l_obj = l_obj.next

```

Listing 4.10: Building the dets list from `NvDsObjectMeta`

4.9.3 SC1State Invocation

```

1 # Run the full SC1 algorithm on this frame
2 persons = state.process_frame(frame_bgr, timestamp, dets)
3
4 # Draw bounding boxes, zone overlays, and counters onto the BGR
  frame
5 sc1_logic.draw_frame(frame_bgr, persons, state.entry_count, state.
  pec_count)
6
7 # Convert BGR -> RGBA and write back into the NVMM buffer
8 frame_rgba_out = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGBA)
9 np.copyto(n_frame, frame_rgba_out) # in-place write to the mapped
  NVMM view

```

Listing 4.11: Business logic call and draw-back

4.9.4 NVMM Buffer Unmap

```

1 try:
2     pyds.unmap_nvds_buf_surface(hash(gst_buffer), frame_meta.
  batch_id)
3 except Exception:
4     pass # some DS versions auto-unmap; absorb double-unmap
  errors

```

Listing 4.12: Releasing the CPU mapping

Releasing the mapping allows the NVMM allocator to reclaim GPU-side cache coherency and permits downstream elements (encoder) to access the buffer.

4.9.5 EOS Injection for Smoke Tests

```

1 eos_sent = {"flag": False} # closure-safe mutable
2
3 if max_frames and stats["timed_frames"] >= max_frames and not
  eos_sent["flag"]:
4     eos_sent["flag"] = True
5     # Must send EOS from the GLib main thread, not the streaming
  thread.
6     # GLib.idle_add schedules the lambda on the main thread's next
  idle cycle.
7     # The "and False" prevents idle_add from repeating the callback
  .
8     GLib.idle_add(lambda: pipeline.send_event(Gst.Event.new_eos())
  and False)

```

Listing 4.13: Thread-safe EOS injection via GLib.idle_add

Why not `pad.send_event()` directly from the probe? A pad probe runs on the *streaming thread* (the GStreamer data-flow thread), not the main GLib thread. Sending EOS via a src-pad from the streaming thread is flagged by GStreamer as “wrong direction” and lets hundreds of additional buffers through before the event propagates. `GLib.idle_add` defers the EOS to the main thread’s event loop, where it is delivered atomically to the pipeline’s sink element.

4.10 GLib Main Loop and Bus

```
1 loop = GLib.MainLoop()
2 bus = pipeline.get_bus()
3 bus.add_signal_watch()
4 bus.connect("message", bus_call, loop)
5
6 pipeline.set_state(Gst.State.PLAYING)
7 loop.run() # blocks until EOS or error
8 pipeline.set_state(Gst.State.NULL)
```

Listing 4.14: Bus and main loop

The GLib main loop processes GStreamer bus messages (EOS, ERROR, WARNING) on the main thread. The `bus_call` function calls `loop.quit()` on EOS or error, triggering clean pipeline shutdown.

5. GPU Subsystem Access

5.1 NVDEC: Hardware Video Decode

Table 5.1: NVDEC access path

Element	<code>nvv4l2decoder</code> (selected by <code>decodebin</code>)
API layer	Video4Linux2 (V4L2) NVMM extension
Input	H.265 bitstream (Elementary Stream)
Output	NV12 frames in <code>memory:NVMM</code>
CPU involvement	Zero — decode is fully in NVDEC silicon
CUDA cores used	Zero during decode

NVDEC output goes directly into an NVMM-allocated NV12 surface. `nvstreammux` and `nvvideoconvert` then operate on this surface in place — no system-RAM copy occurs before inference.

5.2 TensorRT Inference Engine (`nvinfer`)

Table 5.2: TRT engine properties

Property	YOLO11m	OSNet ReID
Source ONNX	77 MB	8.4 MB
TRT FP16 engine	46 MB	5.7 MB
Input shape	$1 \times 3 \times 640 \times 640$	$16 \times 3 \times 256 \times 128$
Output shape	$[N, 6]$ (custom)	$[16, 512]$
Precision	FP16 (Tensor Core)	FP16 (Tensor Core)
Build time (one-off)	≈ 11 min	≈ 2 min

TensorRT uses Tensor Cores for all FP16 GEMM operations (convolution layers are lowered to implicit GEMM). The engine is built once and cached on disk. Subsequent

runs load the serialised plan directly — no ONNX parsing on startup.

5.3 NVMM Zero-Copy Buffer Sharing

```

1  NVDEC (nvv4l2decoder)
2    | allocates NV12 NVMM surface S
3    v
4  nvstreammux
5    | wraps S in NvDsBatchMeta; does NOT copy
6    v
7  nvvideoconvert
8    | converts S in-place NV12->RGBA (GPU kernel, no copy)
9    v
10 nvinfer
11   | reads RGBA surface S as input tensor; runs TRT; attaches
    | metadata
12   | does NOT copy S
13   v
14 nvtracker
15   | reads S for ReID crop extraction (GPU crop kernel)
16   | does NOT copy S
17   v
18 Probe 2
19   | CPU-maps S via pyds.get_nvds_buf_surface()  <-- only CPU touch
20   | reads S (HSV); writes annotated pixels back to S
21   | unmaps S
22   v
23 nvvideoconvert
24   | converts S RGBA->NV12 (GPU kernel, in-place)
25   v
26 nvv4l2h264enc (NVENC)
27   | encodes NV12 surface S; output is compressed bitstream

```

Listing 5.1: Buffer ownership through the pipeline

The same NVMM surface travels the entire pipeline without a single CPU-to-GPU or GPU-to-CPU memory transfer, except for the explicit probe map/unmap.

5.4 CPU Touch Points in the Probe

All CPU-side operations in Probe 2 are on **BGR/RGBA** copies of the cropped **region**, not on the full NVMM surface:

Table 5.3: CPU operations in Probe 2 (per frame)

Operation	Library	Notes
<code>cv2.cvtColor</code> RGBA→BGR	OpenCV (CPU)	Full-frame, 1920×1080×4 bytes
Bounding box crop + HSV conversion	NumPy / OpenCV	Per-detection, sub-region only
<code>pointPolygonTest</code> (zone check)	OpenCV (CPU)	Per detection, negligible
<code>np.hypot</code> (PEC proximity)	NumPy (CPU)	Per detection pair
<code>cv2.cvtColor</code> BGR→RGBA	OpenCV (CPU)	Full-frame write-back
<code>np.copyto</code>	NumPy (CPU)	Full-frame copy to NVMM view

The two full-frame `cvtColor` calls and the `np.copyto` are the dominant CPU cost. At 43 FPS with 8 CPU cores at $\approx 750\%$ utilisation, the Python GIL serialises these per-frame. Future optimisation: convert only the bounding-box sub-regions needed for HSV analysis, skipping the full-frame RGBA↔BGR round-trip.

5.5 DLA: Present, Online, Unused

Table 5.4: DLA status

Cores	DLA0, DLA1
Frequency	614 MHz (<code>jetson_clocks</code>)
Status	Online
Used by pipeline	No

DLA offload for `nvinfer` would require:

1. Rebuild the YOLO11m TRT engine with `-useDLACore=0` and `-allowGPUFallback` in `trtexec` (layers not supported by DLA fall back to GPU automatically).
2. Set `enable-dla=1` and `use-dla-core=0` in `config_infer_primary_yolo11m_sc1.txt`.

DLA offload frees GPU (CUDA core) headroom for additional camera streams — relevant for the multi-stream production target (11 stores).

6. Model Engineering & Compilation

6.1 YOLO11m ONNX Export

YOLO11m is exported to ONNX using the `marcoslucianops/DeepStream-Yolo` export script, which appends a *DeepStreamOutput* post-processing module to the standard Ultralytics ONNX graph.

Why a custom export? Standard YOLO ONNX output is a $[1, 84, 8400]$ tensor ($84 = 4$ box coords + 80 class scores, 8400 anchors). The *DeepStream* custom bbox parser expects a $[N, 6]$ layout per detection: $(x_1, y_1, x_2, y_2, \text{score}, \text{class_id})$. The *DeepStreamOutput* module applies NMS and reformats to this layout inside the ONNX graph, so the TRT engine outputs ready-to-parse detections directly.

Table 6.1: ONNX export parameters

Input shape	$1 \times 3 \times 640 \times 640$ (NCHW)
Output shape	$[N, 6]$ (post-NMS detections)
ONNX opset	17
Dynamic axes	None (static batch for TRT)

6.2 Custom BBox Parser (C++)

`nvdsparsebbox_Yolo.cpp` implements the `NvDsInferParseYolo` function required by the `parse-bbox-func-name` config key.

```
1 extern "C" bool
2 NvDsInferParseYolo(
3     std::vector<NvDsInferLayerInfo> const& outputLayersInfo,
4     NvDsInferNetworkInfo const& networkInfo,
5     NvDsInferParseDetectionParams const& detectionParams,
6     std::vector<NvDsInferParseObjectInfo>& objectList)
7 {
8     // Each detection: output[b*6 + 0..3] = x1,y1,x2,y2
9     //                  output[b*6 + 4]   = score
10    //                  output[b*6 + 5]   = class_id
```

```

11     for (uint b = 0; b < outputSize; ++b) {
12         float maxProb = output[b * 6 + 4];
13         int maxIndex = (int) output[b * 6 + 5];
14         if (maxProb < detectionParams.perClassPreclusterThreshold[
15             maxIndex])
16             continue;
17         addBBBoxProposal(output[b*6], output[b*6+1],
18             output[b*6+2], output[b*6+3],
19             netW, netH, maxIndex, maxProb, binfo);
20     }

```

Listing 6.1: NvDsInferParseYolo signature and core decode loop

The .so is compiled against DS 7.1 headers (/opt/nvidia/deepstream/deepstream-7.1/sources/) and linked into libnvdsinfer_custom_impl_Yolo.so.

6.3 TensorRT FP16 Engine Build

```

1 /usr/src/tensorrt/bin/trtexec \
2     --onnx=models/yolo11m/yolo11m.onnx \
3     --saveEngine=models/yolo11m/yolo11m_b1_gpu0_fp16.engine \
4     --fp16 \
5     --device=0

```

Listing 6.2: Engine build command (one-time, 11 min)

The engine is built on first use of `engine-create-func-name=NvDsInferYoloCudaEngineGet` (implemented in `nvdsinfer_yolo_engine.cpp`), which calls `trtexec` programmatically. On subsequent runs, `nvinfer` detects the `model-engine-file` on disk and loads the serialised plan directly, skipping the build phase entirely.

6.4 OSNet ReID ONNX Export

```

1 from boxmot.reid.backends.pytorch_backend import PyTorchBackend
2
3 WEIGHTS = Path("../osnet_ain_x1_0_msmt17.pt")
4
5 backend = PyTorchBackend(weights=WEIGHTS, device=torch.device("cpu"),
6     half=False)
7 model = backend.model
8 model.eval()
9
10 dummy = torch.zeros(1, 3, 256, 128) # OSNet input: 256x128 RGB
11 with torch.no_grad():
12     out = model(dummy)
13 # out.shape = [1, 512] -- 512-dim L2-normalised feature vector
14
15 torch.onnx.export(model, dummy, str(OUT_ONNX),
16     input_names=["input"], output_names=["output"],

```

```
16     opset_version=17,  
17     dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}})
```

Listing 6.3: `export_osnet_onnx.py` — exports Ahmed’s exact boxmot weights

The exported ONNX has a dynamic batch axis (batch dimension 0), allowing NvMultiObjectTracker to run ReID inference in batches of up to 16 crops per frame (`batchSize=16` in the tracker config).

6.5 Engine Caching and Auto-Naming

Critical implementation detail: NvMultiObjectTracker (`libnvds_nvmultiobjecttracker.so`) does *not* honour the `modelEngineFile` path in the tracker YAML as a literal load path. It auto-derives the cached engine filename from `onnxFile` as:

`<onnxFile>_b<batchSize>_gpu<gpuId>_fp16.engine`

The `modelEngineFile` key must be set to match this derived name exactly or the engine is rebuilt on every pipeline run (≈ 2 min wasted each time).

Correct value: `osnet_ain_x1_0_msmt17.onnx_b16_gpu0_fp16.engine`

7. Configuration Reference

7.1 nvinfer Primary Detector Config

```
1 [property]
2 gpu-id=0
3 net-scale-factor=0.0039215697906911373    # 1/255
4 model-color-format=0                      # RGB input
5 onnx-file=.../yolo11m.onnx
6 model-engine-file=.../yolo11m_b1_gpu0_fp16.engine
7 labelfile-path=.../labels.txt
8 batch-size=1
9 network-mode=2                           # 2 = FP16
10 num-detected-classes=80
11 filter-out-class-ids=1;2;...;79          # person (class 0) only
12 interval=0                               # run inference on every
    frame
13 process-mode=1                           # primary detector
14 network-type=0                           # detector
15 cluster-mode=2                           # NMS
16 maintain-aspect-ratio=1
17 symmetric-padding=1
18 parse-bbox-func-name=NvDsInferParseYolo
19 custom-lib-path=.../libnvdsinfer_custom_impl_Yolo.so
20 engine-create-func-name=NvDsInferYoloCudaEngineGet
21
22 [class-attrs-all]
23 nms-iou-threshold=0.5
24 pre-cluster-threshold=0.1                 # == SC1.py DETECT_CONF
25 topk=300
```

Listing 7.1: configs/config_infer_primary_yolo11m_scl.txt

7.2 NvDCF + ReID Tracker Config

```
1 BaseConfig:
2     minDetectorConfidence: 0.1             # TRACK_LOW_THRESH
3
4 TargetManagement:
```

```

5   maxShadowTrackingAge: 540          # 21.6s at 25fps (=
   TRACK_BUFFER)
6   tentativeDetectorConfidence: 0.2   # NEW_TRACK_THRESH
7
8   DataAssociator:
9     minMatchingScore4Overall: 0.8     # MATCH_THRESH
10    minMatchingScore4VisualSimilarity: 0.3 # APPEARANCE_THRESH
11
12   ReID:
13     reidType: 2                      # ReID-based re-association
14     batchSize: 16                    # crops per inference call
15     reidFeatureSize: 512             # OSNet output dimensionality
16     inferDims: [3, 256, 128]        # OSNet input: C x H x W
17     networkMode: 1                  # FP16
18     inputOrder: 0                    # NCHW
19     colorFormat: 0                   # RGB (boxmot normalises BGR->
   RGB)
20     offsets: [123.6750, 116.2800, 103.5300] # ImageNet mean * 255
21     netScaleFactor: 0.01735207      # ~= 1 / (255 * mean(std))
22     keepAspc: 0                      # direct resize (matches boxmot
   )
23     addFeatureNormalization: 1       # L2-normalise output
24     onnxFile: .../osnet_ain_x1_0_msmt17.onnx
25     modelEngineFile: .../osnet_ain_x1_0_msmt17.onnx_b16_gpu0_fp16.
   engine

```

Listing 7.2: configs/config_tracker_sc1.yml (key sections)

7.3 SC1 Algorithm Parameters

	Parameter	Value	Description
	DETECT_CONF	0.1	YOLO detection threshold (all classes)
	CONF_CLIENT	0.2	Min confidence for CLIENT detections
	SEED_SECONDS	3.0	Bootstrap window (all persons = SELLER)
	SELLER_VOTES	2	Uniform-match frames before locking SELLER
	UNIFORM_BAND_LO	[18,0,51]	HSV lower bound for staff trousers
	UNIFORM_BAND_HI	[49,54,240]	HSV upper bound for staff trousers
	COLOR_TAKE	0.30	Min fraction of pants pixels in colour band
	WHITE_MIN	0.25	Min fraction of white pixels in shirt region
	SELLER_ZONE_FRAC	0.50	Min box overlap fraction for seller zone
	PEC_PROX_DIST	350	Seller-client bbox gap threshold (px)
	PEC_MIN_SECS	8.0	Min engagement duration to count as PEC
	PEC_GRACE_SECS	12	Max client occlusion before closing PEC
	PEC_FOLLOW_DIST	160	Max client position drift (px)
	PEC_EMA	0.5	Position anchor smoothing factor
	ENTER_SIGN	+1	Inward direction sign for entry counting
	ENTER_MIN_STEP	1	Min inward movement per frame (px)

Table 7.1: SC1 configurable parameters

8. Deployment

8.1 Python Environment

The pipeline runs inside a Python 3.10 virtual environment at `~/Desktop/ahmed/venv-ahmed-sc1`, created with `-system-site-packages` to inherit the NVIDIA Jetson-specific PyTorch 2.8.0 wheel (CUDA-enabled, cannot be installed via standard `pip`).

```
1 python3 -m venv --system-site-packages \  
2     /home/wisevision/Desktop/ahmed/venv-ahmed-sc1  
3 source ../venv-ahmed-sc1/bin/activate  
4 pip install boxmot==20.0.0 --no-deps  
5 pip install opencv-python==4.13.0.92 --no-deps  
6 pip install numpy==1.26.4      # must be < 2.0
```

Listing 8.1: Environment creation

8.2 NumPy ABI Compatibility Workaround

Jetson’s PyTorch 2.8.0 is compiled against the NumPy 1.x ABI. `boxmot==20.0.0` and `opencv-python==4.13.0.92` declare `numpy>=2.0` in their package metadata, but this declaration is *overly conservative* — both libraries work correctly at runtime with NumPy 1.26.4. The `-no-deps` flag bypasses `pip`’s dependency resolver for these packages only, allowing the correct NumPy version to coexist.

8.3 pyds Installation

```
1 # pyds 1.2.0 wheel for Python 3.10, aarch64, DeepStream 7.1  
2 pip install pyds-1.2.0-py3-none-linux_aarch64.whl
```

Listing 8.2: `pyds` (DeepStream Python bindings) install

8.4 Engine Build (One-Time)

On first run, `nvinfer` and `NvMultiObjectTracker` build TRT engines:

- YOLO11m FP16 engine: ≈ 11 min (built via `NvDsInferYoloCudaEngineGet`)
- OSNet ReID FP16 engine: ≈ 2 min (built by tracker on first batch)

Both engines are cached on disk. Subsequent cold starts take < 5 s.

8.5 Per-Store Zone Calibration

Each deployment requires a one-time calibration session per camera:

1. Extract a reference frame from the NVR RTSP stream:

```
1 ffmpeg -i "rtsp://user:pass@NVR_IP:554/stream1" \
2       -vframes 1 ref_frame.png
```

2. Open `ref_frame.png` and record pixel coordinates for:

- `ENTREE_ZONE`: polygon covering the doorway
- `SELLER_ZONES`: polygons covering staff positions
- `MIRROR_ZONES`: polygons covering mirror reflections
- `IGNORE_ZONES`: polygons covering static false positives
- `LINE1_START/END`: inward-direction reference line

3. Update the zone arrays in `src/sc1_logic.py` (or, once the per-store config JSON is implemented, in `configs/zones_<store_id>.json`).
4. Run a smoke test: `-max-frames 150` (≈ 6 s of footage) and verify detection and annotation in the output video.

8.6 RTSP Live Adaptation (Pending)

The production pipeline requires replacing `filesrc + decodebin` with a live RTSP source:

```
1 rtspsrc = Gst.ElementFactory.make("rtspsrc", "rtspsrc")
2 rtspsrc.set_property("location", "rtsp://user:pass@NVR_IP:554/
   stream1")
3 rtspsrc.set_property("latency", 200)                # ms buffer
4 rtspsrc.set_property("drop-on-latency", True)        # drop late frames
5 rtspsrc.set_property("buffer-mode", 0)              # auto
6
7 # streammux must be reconfigured for live:
8 streammux.set_property("live-source", 1)
```

Listing 8.3: filesrc replacement for live deployment

A stream-loss watchdog is also required: if no new frame arrives within N seconds (NVR reboot, network drop), the probe detects the stall and restarts the pipeline via `GLib.idle_add`.